

Custom email-from domain	N	N	Y	N	N	Y
'Powered By' removable	N	N	Y	N	Y	Y
Downline (sub-hosts)	N/A	N/A	N/A	N	Y unlimited	Y unlimit
Additional seats	N	N	\$59 / \$49 / \$39 tiered	N	N	\$59 / \$ / \$39 tiered
Dedicated success mgr	N	N	N	N	N	Y

Section 5 — Database Schema (Main App)

PostgreSQL schema for the main application database (Supabase). The RAG service has its own separate schema covered in §6.

All tenant-scoped tables include `tenant_id` with RLS policies. Foreign keys cascade where appropriate. Indexes documented per table.

5.1 Tenancy & Identity

tenants

```
CREATE TABLE public.tenants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug TEXT UNIQUE NOT NULL CHECK (slug = LOWER(slug) AND
  slug ~ '^1[a-z0-9-]{1,28}[a-z0-9]$'),
  display_name TEXT NOT NULL,
  legal_name TEXT NOT NULL,
  tenant_type TEXT NOT NULL
  CHECK (tenant_type IN
  ('platform', 'byo_host', 'sub_host')),
  status TEXT NOT NULL DEFAULT 'onboarding'
  CHECK (status IN
  ('onboarding', 'pending_review', 'active', 'suspended', 'terminated')),
  -- Tier and billing
  tier_id UUID REFERENCES public.tier_definitions(id),
  seat_count INTEGER NOT NULL DEFAULT 1 CHECK (seat_count >=
  1),
```

```

billing_period TEXT NOT NULL DEFAULT 'monthly' CHECK
(billing_period IN ('monthly','annual')),

stripe_customer_id TEXT,

stripe_subscription_id TEXT,

stripe_connect_account_id TEXT,

- Contact info

support_email TEXT,

support_phone TEXT,

mailing_address JSONB,

timezone TEXT DEFAULT 'America/New_York',

- Configuration

ai_mode TEXT NOT NULL DEFAULT 'autonomous'

CHECK (ai_mode IN
('autonomous','draft_only','disabled')),

is_sandbox BOOLEAN DEFAULT FALSE,

- Onboarding stage

onboarding_stage TEXT,

signup_completed_at TIMESTAMPTZ,

review_started_at TIMESTAMPTZ,

activated_at TIMESTAMPTZ,

suspended_at TIMESTAMPTZ,

suspended_reason TEXT,

terminated_at TIMESTAMPTZ,

terminated_reason TEXT,

- Legal acceptance

ica_accepted_at TIMESTAMPTZ,

w9_received_at TIMESTAMPTZ,

- Audit

created_at TIMESTAMPTZ DEFAULT NOW(),

updated_at TIMESTAMPTZ DEFAULT NOW()

);

CREATE INDEX tenants_status_idx ON public.tenants(status);

CREATE INDEX tenants_type_idx ON public.tenants(tenant_type);

ALTER TABLE public.tenants ENABLE ROW LEVEL SECURITY;

```

- Policies vary by role; see implementation

users

```
CREATE TABLE public.users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  auth_user_id UUID NOT NULL REFERENCES auth.users(id) ON  
  DELETE CASCADE,  
  tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON  
  DELETE RESTRICT,  
  email TEXT NOT NULL,  
  display_name TEXT,  
  first_name TEXT,  
  last_name TEXT,  
  auth_provider TEXT CHECK (auth_provider IN  
  ('google', 'microsoft', 'facebook')),  
  - Preferences  
  preferred_persona_slug TEXT,  
  marketing_email_opt_in BOOLEAN DEFAULT FALSE,  
  travel_news_opt_in BOOLEAN DEFAULT FALSE,  
  memory_opt_out BOOLEAN DEFAULT FALSE,  
  notif_preferences JSONB DEFAULT '{}':jsonb,  
  - Status  
  status TEXT NOT NULL DEFAULT 'active'  
  CHECK (status IN  
  ('active', 'suspended', 'deleted')),  
  email_status TEXT DEFAULT 'active'  
  CHECK (email_status IN  
  ('active', 'invalid', 'soft_failing', 'complained', 'unsubscribed_all')),  
  deleted_at TIMESTAMPTZ,  
  - Audit  
  last_signed_in_at TIMESTAMPTZ,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE (tenant_id, auth_user_id)  
);  
  
CREATE UNIQUE INDEX users_tenant_email_unique_idx
```

```
ON public.users(tenant_id, email) WHERE email IS NOT NULL;

CREATE INDEX users_status_idx ON public.users(status);
```

5.1.X Hard-delete protection for tenants

Tenants are soft-terminated, not deleted. The combination of FK ON DELETE RESTRICT (above) and a hard-delete prevention trigger makes accidental hard delete structurally difficult.

```
-- Prevent accidental hard DELETE on tenants. Tenant termination
-- uses status = 'terminated' (per §15.14), not row deletion.
-- Hard delete requires explicit override via a session-level flag.

CREATE OR REPLACE FUNCTION public.prevent_tenant_hard_delete()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = ''
AS $$
BEGIN
    IF current_setting('app.allow_tenant_hard_delete', true) = 'true'
THEN
        RETURN OLD;
    END IF;
    RAISE EXCEPTION 'Hard delete of tenants is not permitted. Use
status = 'terminated' instead. To override (e.g., for compliance
hard-delete after data retention period), execute SET
app.allow_tenant_hard_delete = 'true' in the same transaction.';
END;
$$;

CREATE TRIGGER tenant_prevent_delete_trigger
BEFORE DELETE ON public.tenants
FOR EACH ROW
EXECUTE FUNCTION public.prevent_tenant_hard_delete();
```

Override pattern for legitimate hard delete (GDPR Right-to-be-Forgotten after retention period, or compliance with regulatory order):

```
BEGIN;
SET LOCAL app.allow_tenant_hard_delete = 'true';
DELETE FROM public.tenants WHERE id = '<uuid>';
COMMIT;
```

The SET LOCAL ensures the override is transaction-scoped — it does NOT persist to subsequent transactions or other sessions.

Operators performing legitimate hard delete: 1. Verify retention obligations have been met (audit log review). 2. Document the delete in MEMORY.md with reason and timestamp. 3. Execute with the override pattern shown above. 4. Verify cascading cleanup of related rows (commissions, bookings, etc. — these have ON DELETE RESTRICT so the DELETE will fail until those rows are individually removed or also overridden).

This is intentionally friction-heavy. Hard-delete of a tenant should require deliberate action, not be a side effect of a script or admin tool.

RLS Helper Functions

The platform uses two helper functions in RLS policies. Both are SECURITY DEFINER with a locked search_path to prevent the canonical Postgres privilege-escalation pattern (an attacker creating a same-named table/function in a schema the function might resolve against). Every SECURITY DEFINER function in the platform follows this convention — see §5.1.1 below.

auth_user_in_tenant(target_tenant_id UUID)

Returns TRUE if the authenticated user has an active row in the named tenant. Used in row-level policies that scope SELECT/UPDATE/DELETE to the caller's tenant.

```

CREATE OR REPLACE FUNCTION
public.auth_user_in_tenant(target_tenant_id UUID)
RETURNS BOOLEAN
LANGUAGE SQL
SECURITY DEFINER
STABLE
SET search_path = ''
AS $$
    SELECT EXISTS (
        SELECT 1
        FROM public.users
        WHERE auth_user_id = auth.uid()
              AND tenant_id = target_tenant_id
              AND status    = 'active'
    );
$$;

REVOKE EXECUTE ON FUNCTION public.auth_user_in_tenant(UUID) FROM
public;
GRANT EXECUTE ON FUNCTION public.auth_user_in_tenant(UUID) TO
authenticated;

```

This function does NOT check tenant status. It answers only the question “is this user a member of this tenant?” Use `tenant_is_active` (below) when the policy also needs the tenant to be operational.

tenant_is_active(target_tenant_id UUID)

Returns TRUE if the named tenant is in a state that permits user-initiated writes. Used in tandem with `auth_user_in_tenant` on write policies.

```

CREATE OR REPLACE FUNCTION public.tenant_is_active(target_tenant_id
UUID)
RETURNS BOOLEAN
LANGUAGE SQL
SECURITY DEFINER
STABLE
SET search_path = ''
AS $$
    SELECT EXISTS (
        SELECT 1
        FROM public.tenants
        WHERE id      = target_tenant_id
              AND status = 'active'
    );
$$;

```

```

        REVOKE EXECUTE ON FUNCTION public.tenant_is_active(UUID) FROM
public;
        GRANT EXECUTE ON FUNCTION public.tenant_is_active(UUID) TO
authenticated;

```

Tenants in suspended, terminated, pending_review, or onboarding status return FALSE. This blocks user-initiated writes at the database layer even if the application-layer permission check is skipped or buggy.

System-initiated writes (Stripe webhooks, Inngest jobs running under service-role context) do NOT go through these RLS policies — they bypass RLS by definition. The discipline for service-role usage is defined in §26.3a.

Policy authoring convention

Read policies (SELECT) use `auth_user_in_tenant` alone — a user can read their data from a suspended tenant (e.g., to download historical bookings before termination) but cannot write to it.

Write policies (INSERT, UPDATE, DELETE) use BOTH:

```

-- Example: bookings policy
CREATE POLICY bookings_write_policy ON public.bookings
FOR ALL
USING (auth_user_in_tenant(tenant_id))
WITH CHECK (auth_user_in_tenant(tenant_id) AND
tenant_is_active(tenant_id));

```

The USING clause governs visibility; the WITH CHECK clause governs writes. A user in a suspended tenant can SELECT their historical bookings but cannot INSERT new ones.

5.1.1 Convention for all SECURITY DEFINER functions

Every SECURITY DEFINER function in the platform — including any helper added in future migrations — MUST:

1. Set `SET search_path = ''` in the function definition.
2. Fully qualify every table/function reference (`public.users`, not `users`).
3. Be created in the `public` schema (so the security boundary is clear).
4. `REVOKE EXECUTE FROM public` after creation, then `GRANT EXECUTE` to the specific role that needs it (typically `authenticated`).

Migrations that introduce a SECURITY DEFINER function without these four steps fail the migration lint gate per §30.8.

5.1.2 Required RLS Policy Coverage

Every tenant-scoped table in this section MUST have RLS enabled AND policies covering all four operations (SELECT, INSERT, UPDATE, DELETE). A table with RLS enabled but missing policies for one or more operations is a tenant-isolation hole — Postgres denies the uncovered operation entirely (no rows visible, no writes possible), which engineering will be tempted to “fix” by disabling RLS.

The minimum policy set for every tenant-scoped table:

```

ALTER TABLE public.<table_name> ENABLE ROW LEVEL SECURITY;

CREATE POLICY <table>_select_policy ON public.<table_name>
FOR SELECT
USING (auth_user_in_tenant(tenant_id));

CREATE POLICY <table>_insert_policy ON public.<table_name>
FOR INSERT
WITH CHECK (
    auth_user_in_tenant(tenant_id)
    AND tenant_is_active(tenant_id)
);

CREATE POLICY <table>_update_policy ON public.<table_name>
FOR UPDATE
USING (auth_user_in_tenant(tenant_id))
WITH CHECK (
    auth_user_in_tenant(tenant_id)
    AND tenant_is_active(tenant_id)
);

CREATE POLICY <table>_delete_policy ON public.<table_name>
FOR DELETE
USING (
    auth_user_in_tenant(tenant_id)
    AND tenant_is_active(tenant_id)
);

```

Tables that legitimately deviate from this pattern (e.g., platform-admin tables like `host_adapters`, `legal_documents`, `tier_definitions` that are not tenant-scoped) require explicit documentation in `db/rls-snapshot.sql` (per §30.8) with a comment explaining the exception.

Deviation: read-only-after-suspension behavior

The policy set above lets a user in a suspended tenant `SELECT` their data but blocks writes. This is the default. Tables that should block reads from suspended tenants too (e.g., active conversation context, where reading the suspended tenant’s data is misleading rather than helpful) override with:

```

CREATE POLICY <table>_select_policy ON public.<table_name>
FOR SELECT
USING (
    auth_user_in_tenant(tenant_id)
    AND tenant_is_active(tenant_id)
);

```

The choice is per-table and documented in the table’s DDL comment.

Enforcement

§30.8’s RLS policy snapshot diff is extended to also assert:

- Every table with a `tenant_id` column has RLS enabled.
- Every such table has at least one policy for each of `SELECT`, `INSERT`, `UPDATE`, `DELETE` (or is explicitly listed in the exception file).
- No policy uses `USING (true)` or `WITH CHECK (true)` — these are no-op policies that defeat RLS.

Migrations that violate these rules fail the gate. See §30.8.

5.2 Conversations & Messages

```
CREATE TABLE public.conversations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL REFERENCES public.tenants(id),  
  user_id UUID REFERENCES public.users(id),  
  anonymous_session_id UUID,  
  contact_id UUID REFERENCES public.contacts(id),  
  active_persona_id UUID REFERENCES public.personas(id),  
  title TEXT,  
  status TEXT DEFAULT 'active'  
      CHECK (status IN  
            ('active', 'escalated', 'closed', 'archived')),  
  summary TEXT,  
  summary_updated_at TIMESTAMPTZ,  
  first_message_at TIMESTAMPTZ,  
  last_message_at TIMESTAMPTZ,  
  message_count INTEGER DEFAULT 0,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE TABLE public.messages (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  tenant_id UUID NOT NULL REFERENCES public.tenants(id),  
  conversation_id UUID NOT NULL REFERENCES  
  public.conversations(id) ON DELETE CASCADE,  
  role TEXT NOT NULL CHECK (role IN  
    ('user', 'assistant', 'system', 'tool')),  
  content TEXT NOT NULL,  
  persona_id UUID REFERENCES public.personas(id),  
  tool_calls JSONB,  
  rag_chunks_used JSONB,  
  supervisor_findings JSONB,  
  feedback_score INTEGER CHECK (feedback_score IN (-1, 0, 1)),
```



```

feedback_reason TEXT,

created_at TIMESTAMPTZ DEFAULT NOW()

);

CREATE INDEX messages_conversation_idx ON
public.messages(conversation_id, created_at);

CREATE INDEX conversations_tenant_user_idx ON
public.conversations(tenant_id, user_id);

```

5.3 Bookings & Commissions

Money columns: Every money column in this section is BIGINT cents per §14.0.1. Rates are NUMERIC(5,4) per §14.0.2. Computation goes through the money utility per §14.0.4.

```

CREATE TYPE booking_status AS ENUM (

'draft','submitted','pending_host_review','confirmed','modified','cancelled','sailed','completed'

);

CREATE TYPE commission_status AS ENUM (

'expected','invoiced','received','partial','overdue','disputed','waived'

);

CREATE TABLE public.bookings (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

tenant_id UUID NOT NULL REFERENCES public.tenants(id),

user_id UUID REFERENCES public.users(id),

primary_contact_id UUID REFERENCES public.contacts(id),

group_booking_id UUID REFERENCES public.group_bookings(id),

-- Trip details

booking_type TEXT NOT NULL DEFAULT 'cruise',

cruise_line TEXT,

ship_name TEXT,

sailing_date DATE,

return_date DATE,

duration_nights INTEGER,

departure_port TEXT,

itinerary JSONB,

cabin_category TEXT,

cabin_assignment TEXT,

```

- Host context

host_adapter TEXT,

provider_booking_ref TEXT,

- Financial (see §14.0 for money representation rules)

total_amount_cents BIGINT,

commissionable_fare_cents BIGINT,

currency TEXT NOT NULL DEFAULT 'USD',

fare_breakdown JSONB,

- Status

status booking_status NOT NULL DEFAULT 'draft',

draft_stage TEXT,

submitted_at TIMESTAMPTZ,

confirmed_at TIMESTAMPTZ,

cancelled_at TIMESTAMPTZ,

cancellation_reason TEXT,

- Misc

notes TEXT,

created_at TIMESTAMPTZ DEFAULT NOW(),

updated_at TIMESTAMPTZ DEFAULT NOW()

);

CREATE TABLE public.commissions (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

tenant_id UUID NOT NULL REFERENCES public.tenants(id),

booking_id UUID NOT NULL REFERENCES public.bookings(id),

- Expected (locked at booking submission; see §14.0 for money rules
and §14.3 for the locking rule)

commissionable_fare_cents BIGINT NOT NULL,

commission_rate NUMERIC(5,4) NOT NULL,

platform_split_rate NUMERIC(5,4) NOT NULL,

gross_commission_cents BIGINT NOT NULL,

host_booking_fee_cents BIGINT NOT NULL DEFAULT 0,

host_booking_fee_rule_ref TEXT,

net_commission_cents BIGINT NOT NULL,

platform_retained_cents BIGINT NOT NULL,

```

subhost_payable_cents BIGINT NOT NULL,
currency TEXT NOT NULL DEFAULT 'USD',
expected_payment_date DATE,
- Actual (after statement)
received_commission_cents BIGINT,
payment_received_date DATE,
- Status
status commission_status NOT NULL DEFAULT 'expected',
statement_period DATERANGE,
statement_line_ref TEXT,
- Dispute
dispute_initiated_by TEXT CHECK (dispute_initiated_by IN
('tenant', 'platform', 'host')),
dispute_status TEXT,
dispute_opened_at TIMESTAMPTZ,
dispute_resolved_at TIMESTAMPTZ,
dispute_resolution_notes TEXT,
- Audit
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

```

Subcontractors (Sub-Host Internal Tracking)

Sub-host tenants can record their own subcontractors for internal revenue forecasting. The platform has no relationship with these subcontractors — they're not platform tenants, the platform doesn't pay them, doesn't validate their identities, and doesn't generate tax forms for them. This is a private bookkeeping feature visible only inside the sub-host's tenant.

```

CREATE TABLE public.subcontractors (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES public.tenants(id) ON
DELETE RESTRICT,
name TEXT NOT NULL,
payout_percent NUMERIC(5,2) NOT NULL CHECK (payout_percent
BETWEEN 0 AND 100),
is_active BOOLEAN DEFAULT TRUE,
notes TEXT,

```

```

created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX subcontractors_tenant_idx ON
public.subcontractors(tenant_id);

- Bookings get an optional reference to a subcontractor (set by the
sub-host)

ALTER TABLE public.bookings ADD COLUMN

subcontractor_id UUID REFERENCES public.subcontractors(id);

UI gating: the subcontractor feature only appears for tenants with
tenant_type = 'sub_host'. BYO-host tenants and the platform's own
agency don't see it.

```

Payout Balances & Records

```

CREATE TABLE public.payout_balances (

tenant_id UUID PRIMARY KEY REFERENCES public.tenants(id) ON
DELETE RESTRICT,

pending_cents BIGINT NOT NULL DEFAULT 0,

available_cents BIGINT NOT NULL DEFAULT 0,

in_transit_cents BIGINT NOT NULL DEFAULT 0,

hold_period_days INTEGER NOT NULL,

payout_schedule TEXT CHECK (payout_schedule IN
('weekly', 'biweekly', 'monthly', 'on_demand')),

next_payout_at TIMESTAMPTZ,

minimum_payout_cents BIGINT DEFAULT 5000,

updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE public.payout_records (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

tenant_id UUID NOT NULL REFERENCES public.tenants(id),

amount_cents BIGINT NOT NULL,

stripe_transfer_id TEXT UNIQUE,

status TEXT NOT NULL CHECK (status IN
('processing', 'paid', 'failed')),

statement_period_start DATE,

statement_period_end DATE,

statement_pdf_url TEXT,

```

```

settled_at TIMESTAMPTZ,

failed_at TIMESTAMPTZ,

failure_reason TEXT,

attempt_generation INTEGER NOT NULL DEFAULT 1,

created_at TIMESTAMPTZ DEFAULT NOW()

);

```

- Per H-6 / §7.9a: every Stripe webhook handler MUST insert into this table BEFORE processing the event. ON CONFLICT (stripe_event_id) DO - NOTHING ends processing — the event was already handled.

```

CREATE TABLE public.stripe_webhook_events ( id UUID PRIMARY
KEY DEFAULT gen_random_uuid(), stripe_event_id TEXT NOT NULL
UNIQUE, event_type TEXT NOT NULL, endpoint TEXT NOT NULL
CHECK (endpoint IN ('platform', 'connect')), tenant_id UUID
REFERENCES public.tenants(id), - tenant_id is nullable for platform-
level events (no tenant context). - For Connect events, the resolved
tenant goes here for audit. raw_event JSONB NOT NULL,
signature_verified_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
processing_started_at TIMESTAMPTZ, processing_completed_at
TIMESTAMPTZ, processing_outcome TEXT CHECK (
processing_outcome IN ('success', 'duplicate', 'error', 'unhandled') ),
error_detail TEXT, - Traces which Stripe-side idempotency key (if any)
generated this - event. For transfer events, this links back to
payout_records via - the key format 'payout-{'payout_records.id'}-
gen{'attempt_generation'}' - per §14.7. processed_idempotency_key
TEXT, created_at TIMESTAMPTZ DEFAULT NOW() );

```

```

CREATE INDEX stripe_webhook_events_type_idx ON
public.stripe_webhook_events (event_type, created_at DESC);

```

```

CREATE INDEX stripe_webhook_events_tenant_idx ON
public.stripe_webhook_events (tenant_id, created_at DESC) WHERE
tenant_id IS NOT NULL;

```

- Retention: 90 days for successful events, 1 year for errors. Nightly cron handles cleanup.

Schema continues with: contacts, contact_relationships, quotes, group_bookings, group_members, group_invitations, group_chat_threads, group_chat_messages, personas, tenant_persona_overrides, tenant_branding, host_adapters, tenant_host_configs, host_adapter_calls, escalation_topics, supervisor_alerts, audit_log, notifications, email_log, email_suppressions, legal_documents, legal_consent, platform_revenue, customer_memories, news_articles, destination_images, generated_images, pre_cruise_email_content. Full DDL in repository.

5.4 Database Access Patterns

The schema in §5.1–§5.3 (and the RAG schema in §6) is multi-tenant. Tenant isolation is enforced at three layers — RLS, application-layer permission check, and a typed `TenantContext` boundary at the data-access layer. This subsection defines the third layer, which exists to keep the other two layers honest.

Without this layer, every code path that uses the service-role key silently bypasses RLS, and tenant isolation reduces to “did the developer remember to put `tenant_id` in the WHERE clause.” That has been a recurring bug shape in multi-tenant SaaS history. The `TenantContext` pattern below makes the boundary explicit in TypeScript types and makes raw service-role usage a clearly-named anti-pattern that lint can catch.

5.4.1 The three database client kinds

The platform has exactly three database client patterns. Code at any level must pick one consciously.

Client kind	Use case	Authority
<code>userClient(req)</code>	Any code path serving an authenticated user request	Uses the user’s JWT; subject to RLS
<code>tenantClient(ctx: TenantContext)</code>	Background jobs, webhook handlers, or admin paths that act on behalf of a specific tenant	Service role under the hood, but every query is filtered by <code>ctx.tenant_id</code> in a wrapper
<code>platformAdminClient()</code>	Strictly platform-admin operations crossing tenant boundaries (e.g., the global RAG promotion queue)	Raw service role with no scoping

`userClient` is the default for HTTP handlers. `tenantClient` is the only correct client for Inngest jobs and webhook handlers acting on one tenant. `platformAdminClient` is rare, named, and audited.

5.4.2 The `TenantContext` type

```
// src/lib/db/tenant-context.ts (path indicative)

export type TenantContext = {
  tenant_id: string;
  // Source of the context – recorded for audit. Cannot be derived
  // in code.
  source:
    | { kind: 'http_request'; user_id: string }
    | { kind: 'stripe_webhook'; stripe_event_id: string }
    | { kind: 'inngest_job'; function_name: string; event_id: string }
    | { kind: 'platform_admin'; admin_user_id: string; reason:
string };
};
```

A `TenantContext` carries `tenant_id` plus a *provenance* — where the context was constructed and under whose authority. The provenance is required, audited in the database client wrapper, and prevents “context laundering” (a function receiving a `TenantContext` from one source and using it in a way that doesn’t belong to that source).

5.4.3 The `tenantClient` wrapper

```

// src/lib/db/tenant-client.ts (path indicative)

export function tenantClient(ctx: TenantContext) {
  const supabase = createServiceRoleClient();

  // Wrap from() so every query is automatically scoped.
  return new Proxy(supabase, {
    get(target, prop) {
      if (prop === 'from') {
        return (table: string) => {
          // Tables NOT in the tenant-scoped allowlist still go raw
          // explicit list in db/tenant-scoped-tables.ts.
          if (!TENANT_SCOPED_TABLES.has(table)) {
            return target.from(table);
          }
          // For tenant-scoped tables, transparently inject the
filter.
          return target.from(table).eq('tenant_id', ctx.tenant_id);
        };
      }
      return Reflect.get(target, prop);
    },
  });
}

```

What this gets us:

1. **Every query against a tenant-scoped table is automatically filtered by tenant_id.** A developer cannot forget the filter — the wrapper applies it before the query reaches Postgres.
2. **Raw service-role usage is a separate, named function (createServiceRoleClient()).** It exists but is rare and only invoked inside tenantClient or platformAdminClient. Direct calls from handler code are caught by the lint rule in §29.

5.4.4 The lint rule

A CI lint rule (per §29) enforces:

- createServiceRoleClient cannot be called outside src/lib/db/tenant-client.ts and src/lib/db/platform-admin-client.ts. Any other call site fails the build.
- tenantClient requires a TenantContext argument typed exactly. Constructing a TenantContext requires going through one of four named factory functions, one per provenance kind (see §5.4.5).
- Raw Supabase URL usage is forbidden outside the three client factories.

The lint runs in every PR. The gate is hard (red CI = no merge).

5.4.5 TenantContext factory functions

Constructing a TenantContext is restricted to four named factories, one per provenance kind. This is where the provenance metadata gets attached and where the authority-to-act check happens.

```

// HTTP request – derives tenant from middleware-resolved tenant +
// validates user belongs to it. Called by assertPermission() per
§26.3.
export async function tenantContextFromRequest(req: Request):

```

```

Promise<TenantContext>;

    // Stripe webhook – derives tenant from the event's metadata or by
    // looking up the stripe_customer_id. Validates the event is for a
    // known tenant.
    export async function tenantContextFromStripeEvent(event:
Stripe.Event): Promise<TenantContext>;

    // Inngest job – derives tenant from the job's event payload, which
    // MUST include tenant_id. Jobs that don't have a tenant_id in their
    // event use platformAdminClient instead.
    export async function tenantContextFromInngestEvent(event:
InngestEvent): Promise<TenantContext>;

    // Platform admin – explicit, requires admin user identity and a
    // reason string for audit. The reason is recorded in audit_log.
    export async function tenantContextForPlatformAdmin(
        admin: { user_id: string },
        target_tenant_id: string,
        reason: string,
    ): Promise<TenantContext>;

```

Each factory is responsible for: - Verifying the caller has authority to construct a context for the named tenant_id. - Attaching the right source discriminant. - Recording the context construction to audit_log for the stripe_webhook and platform_admin cases (high-trust paths).

5.4.6 What the application code looks like

```

// HTTP handler
export async function POST(req: Request) {
    const { ctx, user } = await assertPermission(req, {
        resource: 'booking',
        action: 'submit',
    });
    // ctx is a TenantContext sourced from http_request.
    const db = tenantClient(ctx);
    // db.from('bookings') is automatically scoped by tenant_id.
    await db.from('bookings').insert({ ... });
}

// Inngest job
export const handleCommissionPayout = inngest.createFunction(
    { id: 'commission-payout' },
    { event: 'commission.payout.requested' },
    async ({ event }) => {
        const ctx = await tenantContextFromInngestEvent(event);
        const db = tenantClient(ctx);
        // Same scoping discipline as HTTP – same client surface, same
safety.
        const commissions = await db.from('commissions').select('*');
        // ...
    },
);

// Platform admin (rare)
export async function GET(req: Request) {
    const admin = await assertPlatformAdmin(req);
    const db = platformAdminClient(); // No tenant scoping. Audited
heavily.
    const allTenants = await db.from('tenants').select('*');

```



```
// ...
}
```

5.4.7 Calls Worth Flagging

- **The Proxy trick in tenantClient is the architectural centerpiece.** It works because every Supabase query goes through `.from(table)`. If the codebase later adds raw SQL execution paths (`.rpc()`, `.from('arbitrary_sql_function')`), those paths must either be added to the proxy's scoping logic or be moved to `platformAdminClient`.
- **The lint rule is non-optional.** Without it, an engineer in a hurry imports `createServiceRoleClient` directly and the type-system protection is gone. The lint is the enforcement; the type is the design.
- **Provenance metadata is what makes audit useful.** A bug investigation asking “who constructed a `TenantContext` for tenant X at time Y” needs the source field to answer. Skipping the audit on context construction breaks the forensic chain.
- **tenantClient does not invoke RLS** — it uses service role under the hood. Defense in depth: the *application* enforces scoping via the proxy AND the database has RLS as a backstop. Either layer alone is insufficient; both together are the design.

5.4.8 Platform-Admin Audit Wrapper

The `withPlatformAdminAudit` utility is the single supported way to construct a `platformAdminClient`. It manages the per-job audit row that §26.3a.3 requires.

```
// src/lib/db/platform-admin-reasons.ts
// The complete set of allowed reasons for platform-admin
operations.
// Adding a value to this enum is a deliberate act that affects
audit
// query patterns and forensic vocabulary. Reviewed quarterly per
§26.11.
export const PLATFORM_ADMIN_REASONS = [
  // Tenant lifecycle
  'tenant_listing_for_admin_dashboard',
  'tenant_detail_lookup',
  'tenant_status_change',
  'tenant_termination_processing',
  'tenant_suspension_processing',

  // RAG content moderation
  'rag_chunk_promotion_to_global',
  'rag_chunk_demotion',
  'rag_quarantined_content_review',

  // Commission disputes & overrides
  'commission_dispute_review',
  'commission_manual_override',
  'commission_reconciliation_audit',

  // Abuse monitoring
  'abuse_threshold_breach_review',
  'abuse_signal_aggregation',

  // Bug submission triage
```

```

        'bug_submission_review',
        'auto_fix_pipeline_audit',

        // Help docs ingestion
        'help_doc_publishing',
        'help_doc_rag_sync',

        // Privacy & compliance
        'ccpa_export_processing',
        'ccpa_deletion_processing',
        'gdpr_export_processing', // future use, reserved

        // Health and monitoring
        'cross_tenant_health_aggregation',
        'platform_metrics_rollup',

        // Forensics access
        'forensics_log_review',

        // Customer chat caps (per §24.9)
        'customer_chat_cap_override',

        // Feedback settings (per §6.10)
        'feedback_settings_change',

        // Catch-all for emergencies (use sparingly – must be paired with
        // an audit_log.changes.reason_detail string and a MEMORY.md
entry)
        'manual_emergency_intervention',
    ] as const;

    export type PlatformAdminReason = typeof
PLATFORM_ADMIN_REASONS[number];

    // src/lib/db/platform-admin-client.ts (path indicative)

    type PlatformAdminAuditOptions = {
        admin_user_id: string;
        reason: PlatformAdminReason; // Typed enum – see platform-admin-
reasons.ts
        operation: string; // Function name, e.g.,
'platformAdminListTenants'
        request_context?: { // Optional HTTP context for
ip/user_agent recording
            ip?: string;
            user_agent?: string;
            request_id?: string;
        };
        reason_detail?: string; // Required ONLY when reason =
'manual_emergency_intervention'
    };

    type QueryRecord = {
        op: 'select' | 'insert' | 'update' | 'delete' | 'rpc';
        table: string;
        columns?: string[];
        row_count?: number;
        rpc_name?: string;
    };

    export async function withPlatformAdminAudit<T>(  


```

```

options: PlatformAdminAuditOptions,
fn: (
  db: SupabaseClient,
  recordQuery: (q: QueryRecord) => void,
) => Promise<T>,
): Promise<T> {
  // Nested-call detection (per §26.3a.3 "What 'one job' means")
  if (isInsidePlatformAdminAudit()) {
    // Reuse outer context – same db client, same recordQuery
    const outer = currentPlatformAdminAuditContext();
    return fn(outer.db, outer.recordQuery);
  }

  const correlation_id = generateULID();
  const start = performance.now();
  const queries: QueryRecord[] = [];

  const db = createServiceRoleClient();
  const recordQuery = (q: QueryRecord) => queries.push(q);

  let outcome: 'success' | 'error_thrown' = 'success';
  let error_message: string | null = null;
  let result: T;

  // Enforce reason_detail for the emergency escape hatch
  if (options.reason === 'manual_emergency_intervention' &&
!options.reason_detail) {
    throw new Error(
      'withPlatformAdminAudit: reason_detail is required when reason
is ' +
      '"manual_emergency_intervention". This is a deliberate
friction.',
    );
  }

  try {
    result = await runInsidePlatformAdminAudit(
      { db, recordQuery, correlation_id },
      () => fn(db, recordQuery),
    );
    return result;
  } catch (err) {
    outcome = 'error_thrown';
    error_message = err instanceof Error ? err.message :
String(err);
    throw err;
  } finally {
    // Audit row commits regardless of success/failure
    const duration_ms = Math.round(performance.now() - start);
    await writeAuditRow({
      tenant_id: null,
      actor_user_id: options.admin_user_id,
      actor_type: 'admin',
      action: `platformAdmin.${options.reason}`,
      resource_type: 'platform_admin_operation',
      resource_id: null,
      changes: {
        correlation_id,
        operation: options.operation,
        reason: options.reason,
        reason_detail: options.reason_detail ?? null,

```

```

        outcome,
        duration_ms,
        queries,
        error_message,
    },
    context: options.request_context ?? {},
  });
}
}

```

Implementation notes

- `isInsidePlatformAdminAudit()` and `runInsidePlatformAdminAudit()` use Node's `AsyncLocalStorage` (or equivalent) to track whether the current async context is already inside an audit wrapper. This is what makes nesting safe and idempotent.
- The audit row write uses a separate, dedicated database client (NOT db from the wrapped function's scope) so an audit-row write doesn't participate in any transaction the wrapped function may have started. Audit row commits even if the wrapped function's database work was rolled back.
- `recordQuery` is intentionally lightweight — it pushes a small object onto an in-memory array. It does NOT perform any database I/O of its own. Engineers can call it freely without worrying about audit latency dominating the operation.
- `recordQuery` invocation is voluntary, NOT mandatory. A `platformAdmin*` function that uses db without recording any queries produces an audit row with `changes.queries = []`. This is a deliberate design choice: read-only check operations (“does this tenant exist?”, “is this admin permission still valid?”) should not be forced to inflate the audit row. The audit row itself — the fact that the function ran, with what reason, by which admin, at what time — is the primary forensic value. The query summary is a useful enrichment when present but is not required to make the audit row meaningful.

Future engineers may be tempted to make `recordQuery` mandatory via a runtime assertion at audit-write time (“the queries array is empty, throw an error”). DO NOT make this change. It creates friction for legitimate use cases and produces audit-write failures inside finally blocks, which is the worst failure ergonomics in the stack.

- The lint rule in §5.4.4 is extended to forbid `platformAdminClient()` or `createServiceRoleClient()` calls outside this utility's body. The only way to get a `platformAdminClient` is through `withPlatformAdminAudit`. There is no escape hatch.

Reason values are a closed enum

The reason field on `withPlatformAdminAudit` is typed as `PlatformAdminReason`, a union of string literals defined in `src/lib/db/platform-admin-reasons.ts`. Adding a new reason requires:

1. Edit `platform-admin-reasons.ts` to add the value to the enum.
2. Document the new reason's intent in a comment alongside the enum value.
3. Pass code review — see §26.11 quarterly review.

The enum exists for three reasons:

- **Forensic vocabulary stays bounded.** Investigations against `audit_log` filter on `action = 'platformAdmin.<reason>'`. A closed enum keeps that filter predictable.
- **Spelling drift is impossible.** Free-text reasons inevitably produce `tenant_listing_for_admin_dashboard AND tenant_listing_for_dashboard AND tenant_listing` — three variants for one operation. The enum eliminates this.
- **New reasons get reviewed.** Adding a new reason is a PR that a reviewer sees, not a string literal someone types into a handler. This is the discipline that keeps platform-admin operations visible.

The `manual_emergency_intervention` value is a deliberate escape hatch for incident response. When used:

- `reason_detail` is REQUIRED (the type enforces this at the call site — a future small refinement is a discriminated union; for now the runtime check in `withPlatformAdminAudit` enforces it).
- A `MEMORY.md` entry MUST be added in the same PR (or, for during-incident use, by the end of the incident).
- The incident postmortem references the audit row's `correlation_id`.

This escape hatch exists so engineering can act in emergencies without first writing a new enum value. Its use should be rare — target zero in a healthy quarter.

Calling convention

Every `platformAdmin*` function:

1. Calls `assertPlatformAdmin(req)` at entry to verify admin identity.
2. Wraps the rest of the function body in `withPlatformAdminAudit(...)`.
3. Inside the wrapped body, calls `recordQuery(...)` for each meaningful database operation.

A `platformAdmin*` function that does NOT use `withPlatformAdminAudit` is a lint failure (the lint rule sees a function name matching `^platformAdmin` that doesn't contain a `withPlatformAdminAudit` call in its body).

Section 6 — RAG Service Schema

The RAG (Retrieval-Augmented Generation) service runs in a separate Supabase project from the main app. This separation provides isolation: the main app's schema can evolve without affecting RAG; the RAG service can scale independently; security boundaries are explicit.

6.1 Service Separation Rationale

- Independent scaling: RAG vector queries have different load patterns than transactional app queries
- Security isolation: RAG service has its own credentials and access policies

- Schema evolution: changes to RAG don't require main app migrations
- Independent deployment: RAG service can be redeployed without main app downtime

6.2 Tenant Registry (Synced from Main App)

```
CREATE TABLE public.tenant_registry (  
  tenant_id UUID PRIMARY KEY,  
  tenant_type TEXT NOT NULL,  
  status TEXT NOT NULL,  
  synced_at TIMESTAMPTZ DEFAULT NOW()  
);
```

Synced via tenant lifecycle webhook from main app. RAG service is read-only on tenant data; it doesn't manage tenants directly. No hierarchy information is tracked — the platform's structure is strictly two-level (platform global + tenant), so the RAG service only needs to know that a tenant exists and what type it is.

6.3 Authority Scale

Knowledge chunks are scored by authority. Five-tier scale:

Tier	Range	Description
authoritative_override	0.96-1.0	Manual override by platform admin; explicit verification
official	0.85-0.95	From cruise line, port authority, partner press release
tenant_verified	0.70-0.84	Tenant has explicitly verified accuracy
tenant_default	0.55-0.69	Standard tenant-submitted content
low	0.30-0.54	Community-sourced, unverified, or aged content

Pricing chunks auto-cap at 0.55. Manual override (with reason) bypasses the cap.

Tenants can push their tenant-scoped content to 1.0 authority within their own scope. Within their tenant, their override beats everyone else. Doesn't affect other tenants.

6.4 knowledge_chunks Table

```

CREATE TABLE public.knowledge_chunks (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
content TEXT NOT NULL,
content_hash TEXT NOT NULL,
token_count INTEGER,
embedding VECTOR(1536),
- Scope
scope TEXT NOT NULL CHECK (scope IN ('global','tenant')),
tenant_id UUID REFERENCES public.tenant_registry(tenant_id),
- Categorization
category TEXT NOT NULL,
cruise_line_or_supplier TEXT,
ship_or_property TEXT,
destination TEXT,
agent_scope TEXT[],
tags TEXT[],
- Source
source_type TEXT NOT NULL,
source_url TEXT,
source_domain TEXT,
- Authority
authority_auto NUMERIC(3,2) NOT NULL,
authority_manual_override NUMERIC(3,2),
authority_override_reason TEXT,
- Lifecycle
ingested_at TIMESTAMPTZ DEFAULT NOW(),
expires_at TIMESTAMPTZ,
superseded_by_chunk_id UUID REFERENCES
public.knowledge_chunks(id),
- Special flags
contains_pricing BOOLEAN DEFAULT FALSE,
promo_sell_by_date DATE,
promo_sail_by_date DATE,
promo_status TEXT,

```

```

- Status
status TEXT DEFAULT 'approved',
ingested_by_user_id UUID,
approved_by_user_id UUID,
approved_at TIMESTAMPTZ
);

CREATE INDEX knowledge_chunks_embedding_idx
ON public.knowledge_chunks USING ivfflat (embedding
vector_cosine_ops);

CREATE INDEX knowledge_chunks_scope_tenant_idx
ON public.knowledge_chunks(scope, tenant_id);

CREATE INDEX knowledge_chunks_category_idx
ON public.knowledge_chunks(category);

```

6.5 Knowledge Ingestion Queue

```

CREATE TABLE public.knowledge_ingestion_queue (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

- Submission
raw_content TEXT NOT NULL,
raw_source_url TEXT,
raw_source_type TEXT NOT NULL,
raw_metadata JSONB,
submitted_by_user_id UUID NOT NULL,
submitted_by_tenant_id UUID NOT NULL,
submitted_at TIMESTAMPTZ DEFAULT NOW(),
scope_intent TEXT DEFAULT 'tenant',
priority TEXT DEFAULT 'normal',

```

```

- Normalization
normalized_content TEXT,
ai_suggested_category TEXT,
ai_suggested_cruise_line TEXT,
ai_suggested_ship TEXT,
ai_suggested_destination TEXT,
ai_summary TEXT,

```



```

ai_quality_score NUMERIC(3,2),
ai_authority_score NUMERIC(3,2),
ai_relevance_score_global NUMERIC(3,2),
ai_pricing_detected BOOLEAN,
ai_promo_detection JSONB,
- Review
edited_content TEXT,
edited_category TEXT,
edited_authority_override NUMERIC(3,2),
edited_authority_override_reason TEXT,
edited_expires_at TIMESTAMPTZ,
tenant_reviewed_by_user_id UUID,
tenant_reviewed_at TIMESTAMPTZ,
- Global review (per §22 revised model)
global_review_status TEXT,
global_submitted_at TIMESTAMPTZ,
global_reviewed_by_user_id UUID,
global_reviewed_at TIMESTAMPTZ,
auto_flagged_for_global BOOLEAN DEFAULT FALSE,
tenant_global_promotion_objection BOOLEAN DEFAULT FALSE,
tenant_objection_reason TEXT,
- Promotion
promoted_to_chunk_id UUID REFERENCES
public.knowledge_chunks(id),
promoted_at TIMESTAMPTZ,
promotion_scope TEXT,
- Stage
processing_stage TEXT DEFAULT 'submitted',
failure_reason TEXT
);

```

6.6 Retrieval Logging

```

CREATE TABLE public.rag_retrieval_log (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

tenant_id UUID NOT NULL,
user_id UUID,
conversation_id UUID,
persona_id UUID,
query_text TEXT,
filters_applied JSONB,
chunks_returned JSONB, - chunk IDs + scores
top_k_requested INTEGER,
retrieval_latency_ms INTEGER,
outcome TEXT,
outcome_signal_source TEXT,
occurred_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX rag_retrieval_log_tenant_idx
ON public.rag_retrieval_log(tenant_id, occurred_at DESC);

```

6.7 Promo Lifecycle

Promos transition through four states. The state is a function of the current time relative to two dates: `sell_by` (last date a new booking can use the promo) and `sail_by` (last sailing the promo covers).

```

not_started (NOW < sell_by_start, if defined)
|
▼ (sell_by_start reached, or no sell_by_start)
open_to_new
|
▼ (NOW >= sell_by)
closed_to_new ← retrieval filters this OUT for new prospects
|               BUT includes for contacts with existing booking
|
▼ (NOW >= sail_by)
expired ← retrieval excludes entirely

```

Idempotent state reconciliation

The promo state is NOT just a stored field that a cron flips at the right moment. It is a **function of current time** that the cron reconciles. This pattern matters because:

- Cron failures (Inngest outage, DST bugs, late execution, timezone drift) must not leave promos in wrong states.
- Backfill on recovery must reach the correct state regardless of how many transitions were missed.

The implementation:

-- The expected state for a promo at any moment is computable.

```

-- Implementation as a function so it's used by both retrieval and
-- reconciliation.
CREATE OR REPLACE FUNCTION public.expected_promo_state(
    sell_by_start_at TIMESTAMPTZ,
    sell_by_at TIMESTAMPTZ,
    sail_by_at TIMESTAMPTZ
) RETURNS TEXT
LANGUAGE SQL
IMMUTABLE
SET search_path = ''
AS $$
    SELECT CASE
        WHEN NOW() < sell_by_start_at THEN 'not_started'
        WHEN NOW() >= sail_by_at THEN 'expired'
        WHEN NOW() >= sell_by_at THEN 'closed_to_new'
        ELSE 'open_to_new'
    END;
$$;

```

The reconciliation cron (promo-state-reconcile) runs every 15 minutes and updates any drifted state. The knowledge_chunks table requires columns promo_status, promo_status_reconciled_at, sell_by_start_at, sell_by_at, sail_by_at for this — confirm they exist in §6.4 and add if missing.

Drift alerting

A separate Inngest cron (promo-state-drift-alert) runs every hour and counts promos whose promo_status does not match the expected state. If the count exceeds zero for more than 30 minutes, the platform admin is paged. Drift longer than 30 minutes implies the reconciliation cron itself is broken.

Lazy state read fallback

For retrieval (§8.4 /api/retrieve), the scope filter computes the expected state at query time rather than trusting the stored value:

```

WHERE (NOT is_promo) OR public.expected_promo_state(
    sell_by_start_at, sell_by_at, sail_by_at
) IN ('open_to_new', 'closed_to_new')
-- closed_to_new filtered further per §6.9 / §8.4 contact context

```

This is defense in depth: even if the stored promo_status is stale (reconciliation cron failed), retrieval gets the right answer based on current time.

Backfill on recovery

If the reconciliation cron is paused/broken for an extended period and then resumes, the function above naturally reaches the correct state on the next run — it operates on absolute time, not relative state-machine steps. There is no “I missed an event” problem because there are no events; the state is purely a function of time.

Calls Worth Flagging

- **Timezone correctness is the silent killer.** All date columns on promos are TIMESTAMPTZ. NOW() returns UTC. Tenant-facing promo expiry messages render the date in the tenant’s locale, but

internal comparisons are UTC. Mixing locale-naive timestamps into this calculation produces hour-scale drift.

- **The lazy-state-read fallback in retrieval is non-negotiable.** It's the recovery path for any cron failure. Without it, a cron-down weekend has tenants citing expired promos.
- **Drift count > 0 for 30 minutes pages someone.** Don't tune this to be permissive; promo state drift is a customer-visible bug (AI commits to closed promos, retrieves expired ones).

6.8 Pricing Authority Cap

Pricing content is volatile. Auto-cap at 0.55 reduces the risk of stale prices being treated as authoritative. The cap can be manually overridden (with reason) for content where pricing is intentionally part of authoritative communication (e.g., a freshly-confirmed group rate). The override is logged in audit and visible in chunk metadata.

6.9 Retrieval Scope

Retrieval scope: global content + caller's own tenant. No cross-tenant content visibility under any circumstances. The platform's structure is strictly two-level — there are no ancestor tenants to inherit content from.

– Retrieval query scope:

WHERE scope = 'global'

OR (scope = 'tenant' AND tenant_id = caller_tenant_id)

6.10 Customer Feedback Confidence Factor

Customer feedback signals (thumbs up/down on AI responses citing specific chunks) inform a **feedback confidence factor** — a fourth input to retrieval ranking, additive to the existing relevance $\times$ authority $\times$ recency composite.

This is a deliberate departure from earlier draft language that described feedback as modifying chunk `authority_score` directly. The earlier model conflated intrinsic chunk quality (authority) with observed retrieval performance (feedback). The new model keeps these separate:

- **authority_score** — chunk's intrinsic authority. Set by `authority_auto` at ingestion (per §6.3 ingestion quality signals); optionally overridden by `authority_manual_override` for platform-admin curation. NEVER modified by customer feedback.
- **feedback_confidence_factor** — a separate per-chunk value derived from cumulative customer feedback with exponential time decay. Bounded to `[-limit, +limit]` where `limit` is configurable (default 0.05). Recomputed on retrieval (with a short cache).

Schema additions

```
ALTER TABLE public.knowledge_chunks
  ADD COLUMN feedback_signal_count INTEGER NOT NULL DEFAULT 0,
```

```

    ADD COLUMN feedback_weighted_sum NUMERIC(8,4) NOT NULL DEFAULT 0,
    ADD COLUMN feedback_last_recompute_at TIMESTAMPTZ;

CREATE TABLE public.knowledge_chunk_feedback_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    chunk_id UUID NOT NULL REFERENCES public.knowledge_chunks(id) ON
DELETE CASCADE,
    message_id UUID REFERENCES public.messages(id),
    signal_direction TEXT NOT NULL CHECK (signal_direction IN ('up',
'down')),
    -- Raw weight contribution at the moment of capture, before decay.
    raw_weight NUMERIC(4,2) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX kcfe_chunk_time_idx
ON public.knowledge_chunk_feedback_events(chunk_id, created_at
DESC);

```

The four configurable knobs

Stored in the new platform_settings table (per §5.5 below). Modifiable from the platform admin console:

Setting	Default	Range	Effect
feedback_adjustment_limit	0.05	0.0–0.5	Maximum signed adjustment to retrieval score from feedback
feedback_min_signal_count	2	1–100	Minimum signal count within feedback_period_day to trigger any adjustment
feedback_period_days	30	1–365	Lookback window for the minimum signal count
feedback_decay_half-life_days	90	1–365	Time at which a signal's weight has decayed to 50% of its initial value

All four are platform-wide settings; no per-tenant override. Tenants do not see these knobs.

Changes to any setting take effect on the next retrieval (no batch reprocessing — the recomputation function reads current settings each invocation).

Recomputation function

```

CREATE OR REPLACE FUNCTION public.compute_feedback_factor(
    p_chunk_id UUID
) RETURNS NUMERIC
LANGUAGE plpgsql
STABLE
SET search_path = ''
AS $$

```

```

DECLARE
    v_adjustment_limit NUMERIC;
    v_min_signal_count INTEGER;
    v_period_days INTEGER;
    v_halflife_days INTEGER;
    v_recent_signal_count INTEGER;
    v_decay_weighted_sum NUMERIC;
BEGIN
    -- Read current platform settings.
    SELECT value::NUMERIC INTO v_adjustment_limit
        FROM public.platform_settings WHERE key =
'feedback_adjustment_limit';
    SELECT value::INTEGER INTO v_min_signal_count
        FROM public.platform_settings WHERE key =
'feedback_min_signal_count';
    SELECT value::INTEGER INTO v_period_days
        FROM public.platform_settings WHERE key =
'feedback_period_days';
    SELECT value::INTEGER INTO v_halflife_days
        FROM public.platform_settings WHERE key =
'feedback_decay_halflife_days';

    -- Gate: minimum signal count within the lookback period.
    SELECT COUNT(*) INTO v_recent_signal_count
        FROM public.knowledge_chunk_feedback_events
        WHERE chunk_id = p_chunk_id
        AND created_at >= NOW() - (v_period_days || '
days')::INTERVAL;

    IF v_recent_signal_count < v_min_signal_count THEN
        RETURN 0;
    END IF;

    -- Cumulative-with-decay: sum of raw_weight × decay factor.
    SELECT COALESCE(SUM(
        raw_weight * POWER(0.5, EXTRACT(EPOCH FROM (NOW() - created_at))
/ 86400.0 / v_halflife_days)
    ), 0) INTO v_decay_weighted_sum
        FROM public.knowledge_chunk_feedback_events
        WHERE chunk_id = p_chunk_id;

    -- Clamp to [-limit, +limit].
    RETURN GREATEST(-v_adjustment_limit,
        LEAST(v_adjustment_limit, v_decay_weighted_sum *
0.01));

    -- The * 0.01 scaling factor converts raw weighted-sum units into
    -- the adjustment scale. Calibrate during Phase 1.
END;
$$;

```

Retrieval integration

The retrieval scoring in §8.4 changes from:

$\text{composite_confidence} = \text{match_score} \times \text{authority} \times \text{recency}$

to:

$\text{composite_confidence} = (\text{match_score} \times \text{authority} \times \text{recency}) + \text{feedback_factor}$

where `feedback_factor` comes from `compute_feedback_factor(chunk_id)`, clamped to the configured limit. The retrieval response per §8.4 returns `feedback_factor` as a fifth scoring component for transparency.

Where feedback events come from

The `/api/chat/feedback` endpoint (per §7.2) records thumbs-up/thumbs-down on a message. When the message had a non-empty `cited_chunk_ids` set, each cited chunk gets a `knowledge_chunk_feedback_events` row inserted:

- Thumbs up → `signal_direction = 'up'`, `raw_weight = +1.0`
- Thumbs down → `signal_direction = 'down'`, `raw_weight = -1.0`

A single feedback event can produce multiple feedback rows (one per cited chunk).

Caching

`compute_feedback_factor` is STABLE (deterministic for a single transaction). For high-traffic retrieval, the function result is cached at the application layer per chunk for 5 minutes.

Platform admin surface

The platform admin console gains a “Feedback Tuning” page showing current knob values (editable), histograms of `feedback_factor` distribution, top chunks by absolute `feedback_factor`, and settings-change history. Settings changes are gated by `withPlatformAdminAudit` with reason = `'feedback_settings_change'`.

platform_settings table

New table for platform-wide configurable settings, modifiable from the platform admin console without code deploys:

```
CREATE TABLE public.platform_settings (
  key TEXT PRIMARY KEY,
  value JSONB NOT NULL,
  description TEXT,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by_user_id UUID REFERENCES public.users(id)
);

-- Seed values
INSERT INTO public.platform_settings (key, value, description)
VALUES
  ('feedback_adjustment_limit', '0.05'::JSONB,
   'Max signed adjustment to retrieval score from feedback (per
  §6.10)'),
  ('feedback_min_signal_count', '2'::JSONB,
   'Minimum signal count to trigger any feedback adjustment (per
  §6.10)'),
  ('feedback_period_days', '30'::JSONB,
   'Lookback window for minimum signal count check (per §6.10)'),
  ('feedback_decay_half-life_days', '90'::JSONB,
   'Half-life for exponential decay of older feedback signals (per
  §6.10));
```

This table lives in the main app’s Supabase project (not the RAG project) and is queried by the RAG service via the existing internal API where needed.

Calls Worth Flagging

- **Authority and feedback are separate inputs by design.** A future refactor that “simplifies” by collapsing them into a single score loses this separation. Don’t.
- **Default decay halflife of 90 days is slow.** A chunk that gets consistent thumbs-down for a month will accumulate signal but the adjustment grows gradually. Faster halflife (e.g., 30 days) makes the loop more reactive at the cost of being more thrash-prone. Tune in Phase 1.
- **Minimum signal count = 2 is permissive.** Two thumbs is enough to start moving the factor. This is intentional during early operation when feedback volume is low. Raise as traffic grows.
- **The clamp at ±0.05 is a hard ceiling on impact.** Even thousands of thumbs-down on one chunk only adjusts retrieval score by 0.05. This protects against coordinated-thumbs-down attacks while still letting genuine signal accumulate.
- **Per-tenant override is deliberately NOT supported.** Tenants can’t tune their own corpus’s feedback dynamics. This keeps platform retrieval behavior consistent and prevents tenants from gaming their own scoring.

6.11 PII Exclusion

RAG content must not contain customer PII. Ingestion pipeline (\$22.4) redacts PII before chunks are created. Reports of PII slipping through trigger immediate chunk removal.

6.12 Retention

- Approved chunks: indefinite while tenant is active
- Rejected/failed queue items: 90 days
- Retrieval log detail: 90 days, then aggregated
- Audit log on chunk changes: 7 years

Section 7 — API Routes

The platform exposes ~160 API routes. This section provides a representative summary, grouped by domain. Complete route documentation is generated from code.

7.1 Authentication & Identity

Method	Path	Purpose
GET	/api/auth/callback	OAuth callback handler Complete signup

POST	/api/auth/signup/complete	with profile
POST	/api/auth/signout	Sign out current session
GET	/api/auth/me	Current user, tenant, roles, permissions
GET	/api/auth/consent-status	Returns missing required consents
POST	/api/auth/consent	Record consent for a document version
GET	/api/legal/:doc-type/current	Current version of legal document
POST	/api/auth/transfer-session	Transfer anonymous session

7.2 Chat & AI

Method	Path	Purpose
POST	/api/chat	Send message; returns SSE stream
GET	/api/chat/conversations	List user's conversations
GET	/api/chat/conversations/:id	Get conversation detail
POST	/api/chat/conversations/:id/persona	Switch active persona
POST	/api/chat/feedback	Thumbs up/down on a message
POST	/api/chat/escalate	Customer requests human

7.3 CRM & Quotes

Method	Path	Purpose
GET	/api/crm/contacts	List contacts (filtered, paginated)
POST	/api/crm/contacts	Create contact
PATCH	/api/crm/contacts/:id	Update contact
GET	/api/crm/contacts/:id/timeline	Activity timeline
POST	/api/quotes	Create quote
POST	/api/quotes/:id/send	Send quote to customer
POST	/api/quotes/:id/accept	Customer accepts quote

7.4 Bookings

Method	Path	Purpose
POST	/api/bookings/draft	Create draft booking
GET	/api/bookings/:id	Get booking

PATCH	/api/bookings/:id	Update draft
POST	/api/bookings/:id/submit	Submit to host adapter
POST	/api/bookings/:id/cancel	Cancel
POST	/api/bookings/:id/modify	Modify confirmed booking

7.5 Groups

Method	Path	Purpose
POST	/api/groups	Create group
GET	/api/groups/:id	Group detail
POST	/api/groups/:id/members	Add invitees
POST	/api/groups/:id/broadcast	Message all invitees
GET	/api/groups/invite/:token	Token-gated invitee landing
POST	/api/groups/invite/:token/rsvp	Submit RSVP

7.6 RAG & Knowledge

Method	Path	Purpose
POST	/api/rag/submit	Submit knowledge item
POST	/api/rag/batch-submit	Submit up to 100 items
GET	/api/rag/queue	List tenant's pending queue
POST	/api/rag/queue/:id/approve	Approve item
GET	/api/rag/chunks	List tenant's chunks
PATCH	/api/rag/chunks/:id	Edit chunk

7.7 Commissions & Payouts

Method	Path	Purpose
GET	/api/commissions	List tenant's commissions
GET	/api/commissions/:id	Commission detail
GET	/api/payouts/balance	Current payout balances
GET	/api/payouts/history	Past payouts with statements
POST	/api/payouts/manual	Initiate manual payout (on-demand)

7.8 Webhooks

Method	Path	Source
POST	/api/webhooks/stripe	Stripe events (subscriptions, Connect)
POST	/api/webhooks/resend	Email delivery, bounces, complaints
POST	/api/webhooks/gmail-pubsub	Gmail inbox notifications

7.9 Conventions

- Rate limiting: per-user (100/hour default), per-tenant, per-IP (anonymous)
- Idempotency: critical mutations accept Idempotency-Key header (24-hour cache)
- Streaming: chat uses Server-Sent Events (SSE) with Last-Event-ID for reconnect
- Authorization: every protected route runs `assertPermission()` at handler entry
- Audit logging: significant operations write to `audit_log` table

7.9a Stripe Webhook Handler Contract

Every Stripe webhook handler — `/api/webhooks/stripe/platform` and `/api/webhooks/stripe/connect` — follows this contract. Failure to follow it produces duplicate-processing bugs that, for payment events, move real money twice. The contract is enforceable: integration tests per §30.8 verify a same-event-id delivered twice produces exactly one side-effect.

Step sequence

1. **Verify Stripe signature** using the endpoint-specific webhook secret per §28.7. Reject with 400 on invalid signature; Stripe will not retry.
2. **Atomic dedup insert** into `public.stripe_webhook_events`:

```
const { data, error } = await db
  .from('stripe_webhook_events')
  .insert({
    stripe_event_id: event.id,
    event_type: event.type,
    endpoint: endpointName, // 'platform' | 'connect'
    raw_event: event,
    processing_started_at: new Date().toISOString(),
  })
  .select('id')
  .single();

if (error?.code === '23505') {
  // Unique violation on stripe_event_id – event already
  // processed.
  // Return 200 OK to Stripe; do nothing else.
  return new Response('OK', { status: 200 });
}
if (error) {
  // Different error – return 5xx so Stripe retries.
  return new Response('error', { status: 500 });
}
```

3. **Process the event** in a function-level try/catch. The processing logic varies by event type — commission record updates, subscription state changes, transfer reconciliation, etc.

4. **Mark completion** by updating the row with `processing_completed_at`, `processing_outcome`, and (for failures) `error_detail`. This is the audit record of “yes we handled this.”
5. **Return 200** to Stripe. Stripe will not retry a 200-OK event.

Why the dedup insert is FIRST (not after signature verification)

Signature verification rejects forged events; dedup catches legitimate retries from Stripe. These are independent concerns. The order matters: signature first (cheap, syntactic), dedup second (transactional, before any business logic).

Why “Return 200 on duplicate” rather than “return 5xx and let Stripe retry forever”

A duplicate Stripe event is Stripe doing its job — retrying after a network blip on our side. Returning 200 stops the retry loop. We already have the event recorded.

Linking transfers back to payouts

For events from §14.7 (Stripe Connect transfers), the `processed_idempotency_key` column is populated from the event’s metadata. The format `payout-{payout_records.id}-gen{attempt_generation}` lets reconciliation queries link:

```
-- "Did the transfer for payout X succeed?"
SELECT pr.id, pr.status, swe.event_type, swe.created_at
FROM public.payout_records pr
LEFT JOIN public.stripe_webhook_events swe
  ON swe.processed_idempotency_key =
    'payout-' || pr.id || '-gen' || pr.attempt_generation
WHERE pr.id = '<payout_id>'
ORDER BY swe.created_at;
```

Handlers MUST be idempotent in their own right

The dedup table catches Stripe-side retries. It does NOT catch our own bugs (handler crashes after partial state mutation; on retry the row exists but `processing_outcome` is unset). Handlers should be designed to be safely re-runnable, treating `processing_completed_at IS NULL` as “may need to re-run” and recovering gracefully.

A reconciliation Inngest job (`stripe-webhook-incomplete-reconcile`, runs every 15 minutes) finds rows with `processing_started_at` more than 5 minutes ago and `processing_completed_at IS NULL`, and either re-runs the handler or escalates to platform admin per the event type.

Retention and cleanup

- `processing_outcome = 'success'`: retain 90 days then delete.
- `processing_outcome = 'error'`: retain 1 year (investigation history).
- `processing_outcome = 'duplicate'`: retain 30 days (mostly noise).
- `processing_outcome = 'unhandled'`: retain 1 year (these are event types we received but had no handler for — surface for review).

Cleanup cron runs nightly.